ITU-MiniTwit

DevOps, Software Evolution and Software Maintenance
BSc Group F (gruppe 1)

Sebastian Holt Svendsen <sesv@itu.dk>
Andreas Nygaard Schiøtt <anys@itu.dk> Member Three <xxxx@itu.dk>
Member Four <xxxx@itu.dk> Member Five <xxxx@itu.dk>

May 12, 2026

Word count:
1379 (errors:2)

Contents

1 System's Perspective **2**
1.1 Architecture and Design . . . 2
1.2 Dependencies . . . 4
1.3 Current State of the Systems . . . 4

2 Process' Perspective **5**
2.1 CI/CD Pipeline . . . 5
2.2 Monitoring . . . 7
2.3 Logging . . . 8
2.4 Security Hardening . . . 9
2.5 Availability and Scaling . . . 10

3 Reflection Perspective **11**
3.1 Evolution and Refactoring . . . 11
3.2 Operation . . . 11
3.3 Maintenance . . . 11
3.4 DevOps Style of Work . . . 11

4 Use of Generative AI **12**
4.1 Tools Used . . . 12
4.2 Impact on Work Process . . . 12
ITU-MiniTwit GitHub Repository

1 System's Perspective

Authors: Sebastian Holt Svendsen

In this section, we provide an overview of the ITU-MiniTwit system from a technical perspective. We cover three areas: the architecture and design (section 1.1), the dependencies and technologies used (section 1.2), and the current state of the system (section 1.3). The app is deployed on DigitalOcean as a Docker Swarm cluster, fronted by two nginx load balancers, with a shared Redis for user-session-state and a managed PostgreSQL database for persistent data.

1.1 Architecture and Design

Figure 1 shows the production topology of ITU-MiniTwit deployed on DigitalOcean.

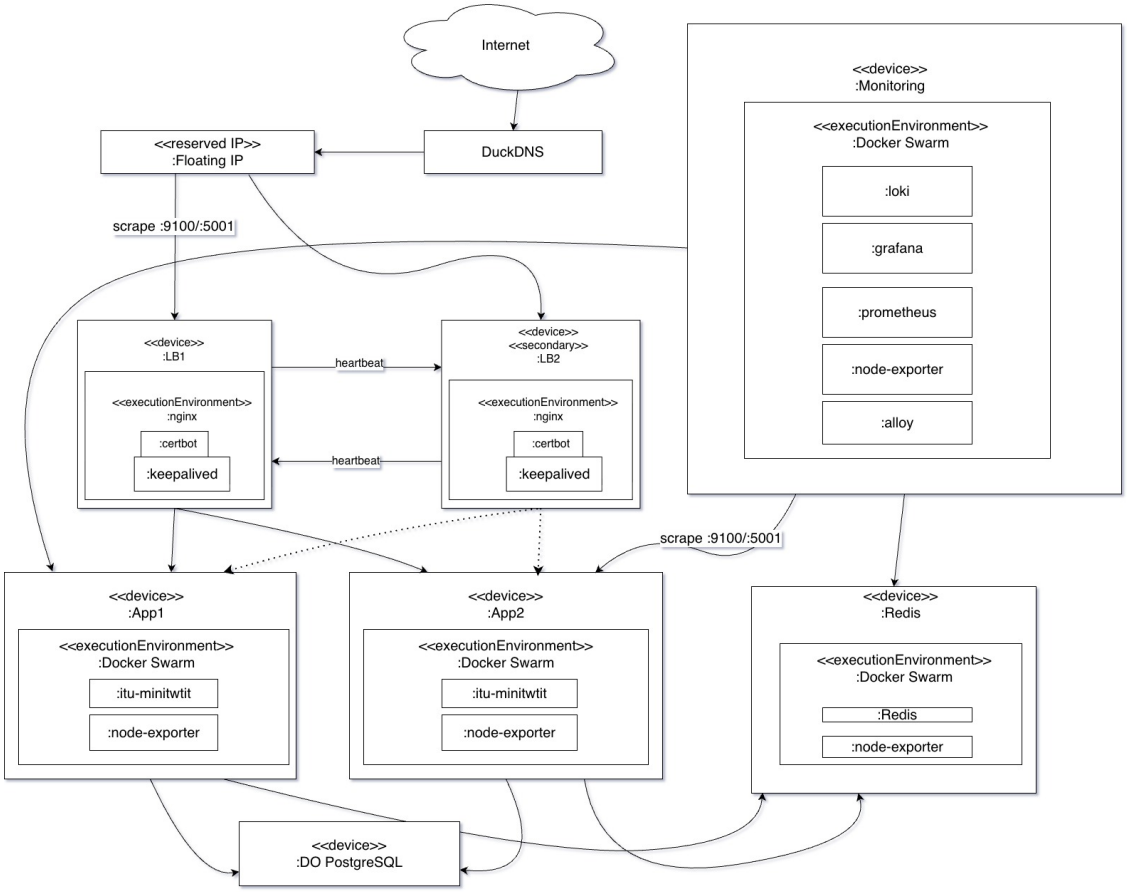

Figure 1: Deployment diagram — production topology of ITU-MiniTwit deployed on DigitalOcean.

We will be going through each tier of the diagram from top-to-bottom.

Tier 1: Public entry. Traffic enters from the internet via the DuckDNS domain. DuckDNS is a free dynamic DNS service for setting up a domain for ITU-MiniTwit. The domain points to a floating IP address, which is a public IP that can be moved between different servers in the cluster, specifically the two load balancers.

Tier 2: Load balancers are two nginx instances, LB1 and LB2, that distribute incoming requests to the webapp instances in the next tier. Two are deployed for higher availability. *Certbot* is used to automatically obtain and renew TLS certificates for the domain, ensuring

secure HTTPS connections. *Keepalived* implements the heartbeat mechanism: LB1 runs as MASTER and LB2 as BACKUP under VRRP. If LB1 stops responding, keepalived on LB2 promotes itself and the Floating IP is reattached automatically, so the public hostname keeps resolving to a healthy node without manual intervention.

Tier 3: App and data contains two app instances, the Redis instance, and the PostgreSQL database. The app instances, App1 and App2, are each a device running a Docker Swarm execution environment that hosts one `itu-minitwit` replica and a `node-exporter`. Docker Swarm schedules the replicas onto nodes labelled `role == app`. Both replicas share two backing stores: a managed PostgreSQL database for persistent data, and a Redis instance on its own node holding session state and ASP.NET DataProtection keys. Because no per-request state is kept inside the app containers, any replica can serve any user, so the load balancers can distribute requests freely and fail a node out without dropping sessions. Rolling updates similarly bring up a new container before stopping the old one, so no one is logged out. Crashed containers are restarted automatically by Swarm's `on-failure` restart policy.

Tier 4: Monitoring is its own device with its own Docker Swarm execution environment running Prometheus, Grafana, Loki, Alloy, and node-exporter. Prometheus scrapes metrics from every node — `:9100` for node-exporter (host metrics) and `:5001` for the app's `/metrics` endpoint (HTTP and runtime metrics). Grafana Alloy runs on every node, reads the local docker socket, and ships container logs to Loki, which stores and indexes them. Grafana queries Prometheus and Loki to render the monitoring and logging dashboards. Keeping the observability stack on its own node means an outage of the app tier does not also bring down monitoring, which would be needed to diagnose it.

Application architecture. Beyond the deployment topology, the application itself follows a layered design split into three .NET projects with a one-way dependency rule. `Chirp.Core` holds the domain entities (`Author`, `Cheep`, `Follows`, `Recheep`). `Chirp.Infrastructure` owns persistence, the EF Core `DbContext`, repositories, and services behind interfaces (`ICheepRepository`, `ICheepService`, etc.), and the database migrations. `Chirp.Web` is the presentation layer: ASP.NET Core Razor Pages for the user interface, scaffolded ASP.NET Identity pages for authentication, and a `MiniTwit-Stub` controller exposing the REST API consumed by the course simulator.

To complement this static view, fig. 2 traces what happens at runtime when a user requests the public timeline.

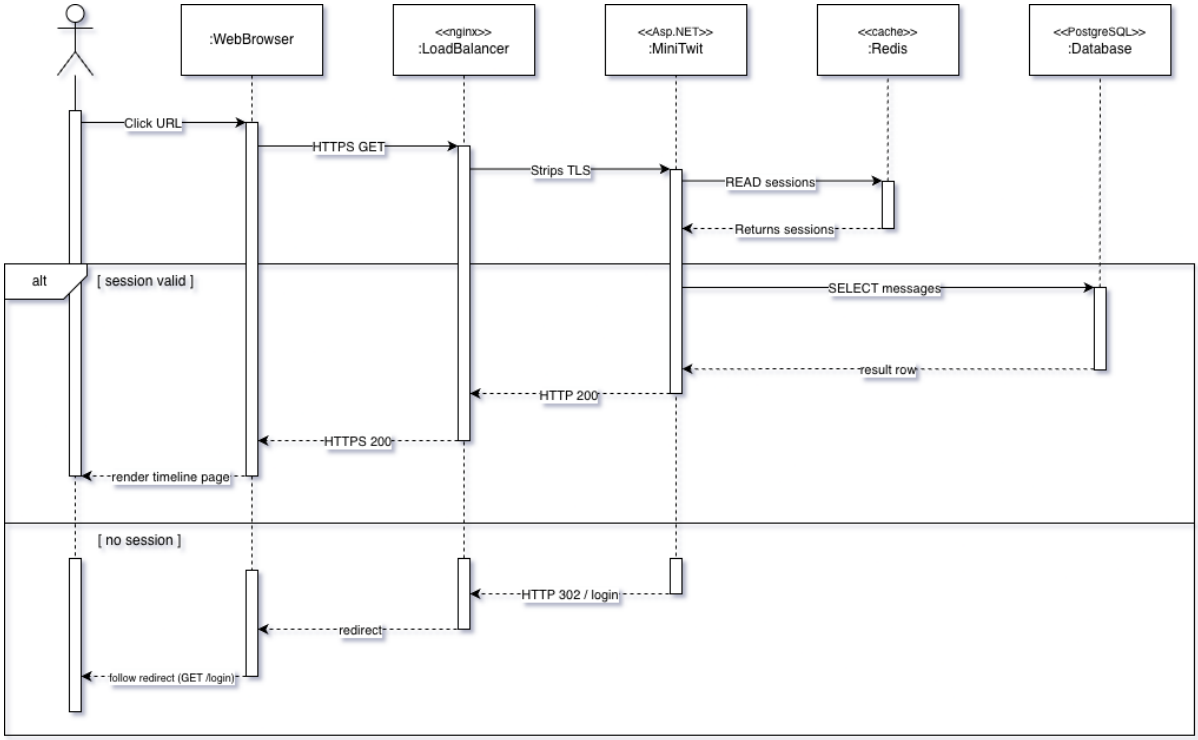

Figure 2: Sequence diagram — request flow to the public timeline.

In the common case the browser issues an `HTTPS GET`, which the active nginx instance terminates and forwards over HTTP to one of the app replicas. The replica reads the caller's session from Redis. With a valid session it queries PostgreSQL for the latest cheeps and returns the rendered timeline page with `HTTP 200` back through nginx to the browser. The diagram makes each tier's responsibility explicit: nginx only terminates TLS, the app holds no per-request state, and Redis and PostgreSQL hold all shared state.
The `alt` fragment shows the alternative flow: when no valid session exists, the replica responds with `HTTP 302 /login`. The browser follows this redirect transparently and re-enters the same flow against `/login`, so the user simply lands on the login page.

1.2 Dependencies

Table 1: Technology dependencies

Technology	Purpose
Docker	Containerisation of all services

1.3 Current State of the Systems

2 Process' Perspective

Authors: Andreas Schiøtt

2.1 CI/CD Pipeline

Table 2: CI/CD tools

Technology	Purpose
GitHub Actions	CI/CD automation
Playwright	end-to-end testing
SonarCloud	Static analysis and code coverage
Hadolint	Dockerfile linter
ShellCheck	Shell script linter
Docker Build	Build and push container images to Docker Hub
Docker Swarm	Automated docker container orchestration
rsync	Idempotent Sync files to the remote swarm manager
Grafana	Logging / monitoring running Prometheus, loki, Alloy
Prometheus	Metric collection and monitoring
Loki	Log aggregation system
Grafana Alloy	Log and monitoring collection

Before we had a running cloud hosted droplet we only ran CI to run a docker container and run unit test on a locally hosted sqlite based application.

After cloud hosting on **DigitalOcean** was setup we added the CD pipeline to the CI pipeline. The CI/CD pipeline were both triggered concurrently on push to main branch which then builds a **Docker image**, pushes it to **Docker Hub** and then deploys it to the **DigitalOcean** droplet using **SSH & docker pull**. With the later focus on idempotent deployment, we added a check to see if the CI pipeline succeeded before the CD pipeline is triggered.

The CI pipeline was extended to include end-to-end testing using **Playwright**, and static code analysis using **SonarCloud**, **Hadolint** and **ShellCheck**. As shown in fig. 3, on every push the pipeline detects which paths changed, builds the project, and then runs static analysis, unit tests, SonarCloud analysis, and end-to-end tests in parallel.

While writing the report we itterativly extended the CI pipeline to check which files were changed to decicde which steps to run, if only the report directory was changed we only run "build PDF" report" step and vice versa.

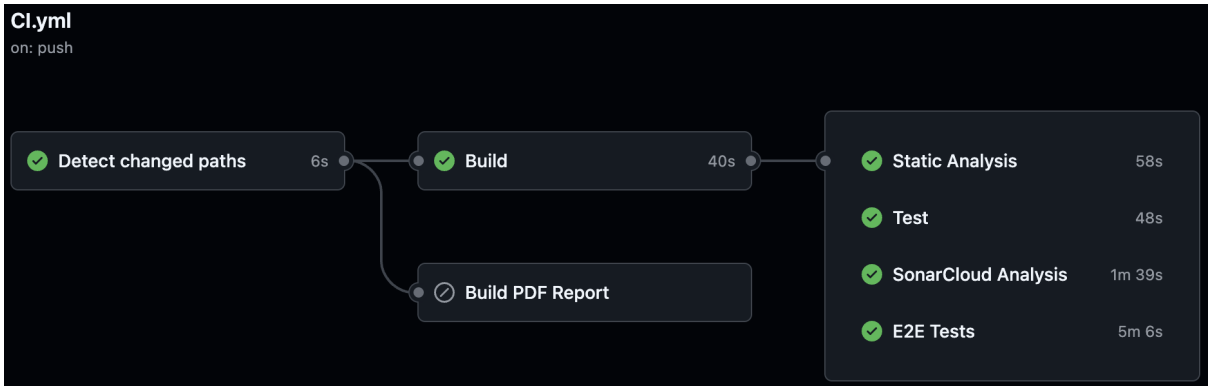

Figure 3: CI pipeline (`CI.yml`).

The CD pipeline is triggered only after a successful CI run, ensuring that broken builds are never deployed. As shown in fig. 4, it consists of three sequential jobs: building and pushing the Docker image (*build-push*), deploying the application (*Deploy*), and deploying the monitoring stack (*Deploy-Monitoring*) with **Grafana**, **Prometheus**, **Loki** and **Grafana Alloy**.

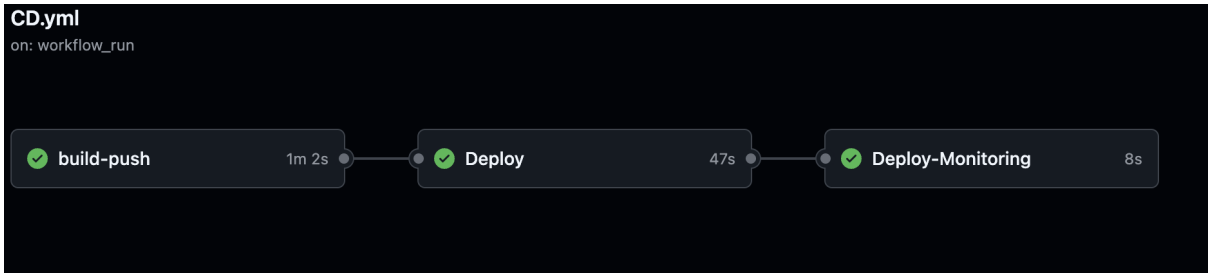

Figure 4: CD pipeline

To ensure high availability of the application, we extended the CD pipeline to deploy the application using **Docker Swarm**, which deploys the application as a swarm with multiple replica nodes while a load balencer distributes traffic across the replicas. this deployment uses rolling updates, which starts a new docker container before closing the previous one to avoid downtime. The application also uses shared **Redis** which caches data across the replicas. The detailed steps of the *Deploy* job are shown in fig. 5:

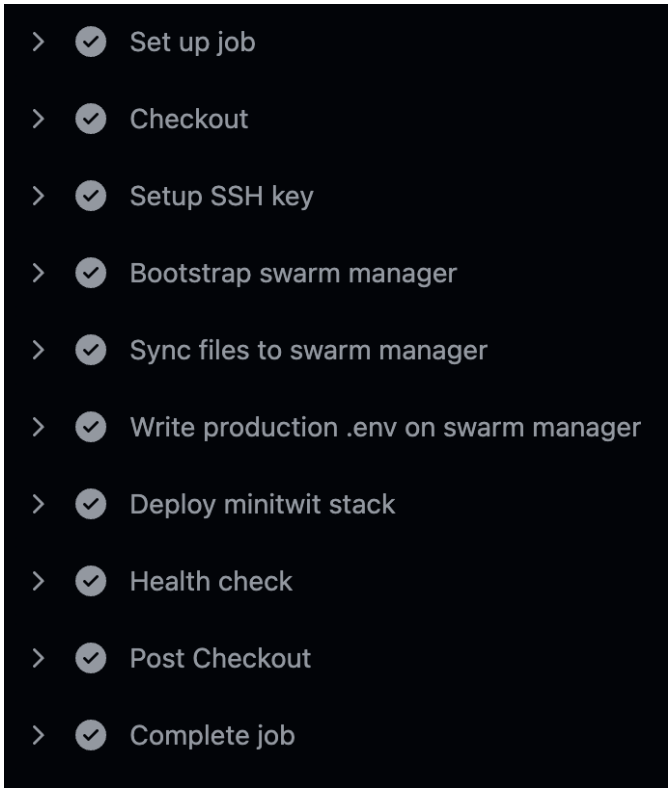

Figure 5: Detailed steps of the *Deploy* job.

2.2 Monitoring

Link to Grafana monitoring dashboard
Metrics are collected through **Prometheus** and visualised in **Grafana**, both running as Docker services on a dedicated monitoring droplet, isolated from the application. Prometheus scrapes application metrics via `'itu-minitwit-app'` defined in the Prometheus configuration and metrics from Node Exporter running on each Swarm node.

We have itterativly upated which data is collected and visualised in the Grafana dashboard based on the needs of the project. At the projects end we have the Grafana dashboard organised into five sections, as shown in fig. 6:

- **Health overview** — application up/down status, average response time, requests / sec, in-flight requests, error rate and current memory usage.
- **HTTP traffic** — request rate by status code, ... response-time percentiles grouped by p50 p90 and p99, request rate /s grouped by HTTP method, and the response time grouped by api endpoints.
- **Error tracking** error rates req/s grouped by 400/500 http response, errors mesured in req/s per endpoint.
- **Process & runtime** — process CPU usage, process memory usage, thread count, garbage colllection rate.
- **Uptime & Availability** — uptime & availability over time (bar graf), Availability mesured in percentage over the selected time period.

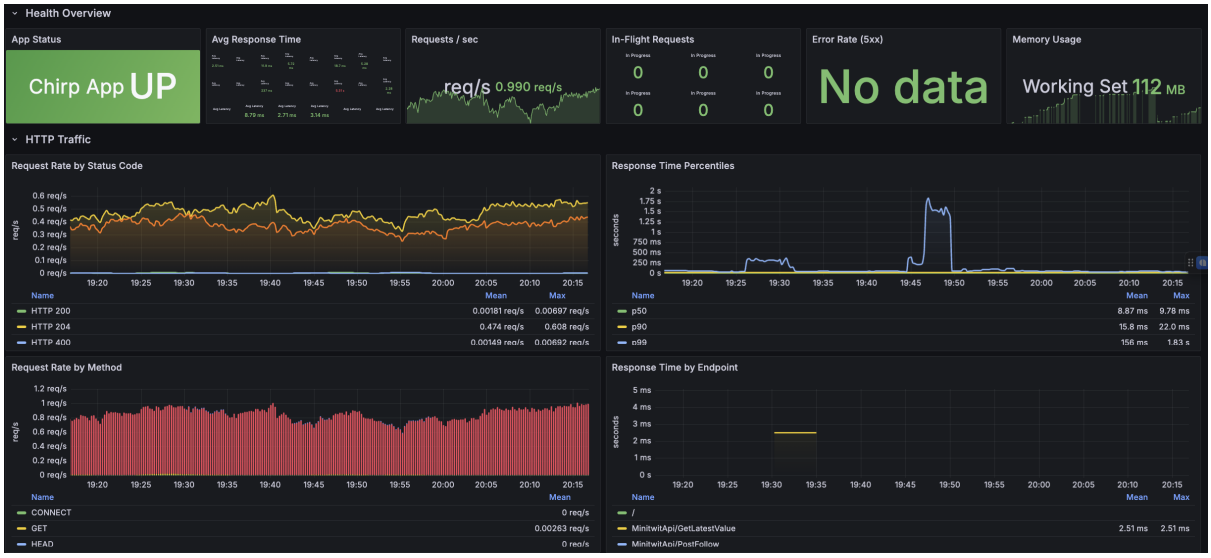

Figure 6: Grafana monitoring dashboard.

2.3 Logging

Link to Grafana logging dashboard

Application logs are aggregated using **Loki** and collected by **Grafana Alloy**. Each line in the logs is labelled with the container name, service name, stream, and image name. Grafana Alloy collects logs from all the replica nodes and forwards them to Loki for aggregation and storage. **Loki** rejects old samples above the "age" of 168hours (7 days) to prevent unbounded log storage growth.

The application logs the following events:

- HTTP requests — method, path, response status code, and elapsed time mesured in ms.
- Unhandled exceptions — method, path, error details.
- Custom logs through `ILogger` throughout the minitwit codebase.

The logs are visualised in a Grafana dashboard with the following panels:

- **Total Log lines & error lines** — Total log lines & errors within the selected time period
- **Log lines & Errors per container** — Log lines & errors grouped by container within the selected time period

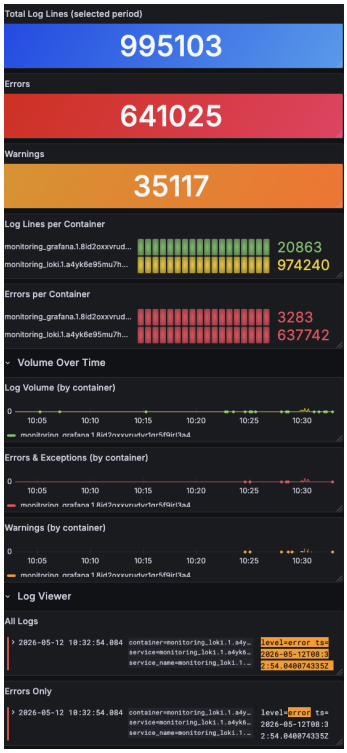

Figure 7: Loki log aggregation in Grafana.

2.4 Security Hardening

measures applied to harden the system:

- **GitHub Secrets** — database connection strings and API keys are stored as encrypted secrets and injected at deploy time.
- **Firewall** — inbound and outbound traffic on both application droplets and the database is restricted to only the ports and sources required.

Static code analysis is integrated in the CI pipeline using **SonarCloud**. Every build is analysed for bugs, coverage, code duplication and security vulnerabilities. As shown in fig. 8, there are still a number of security vulnerabilities but due to a lack of group commitment the active group members didn't have the time to fix them before the deadline.

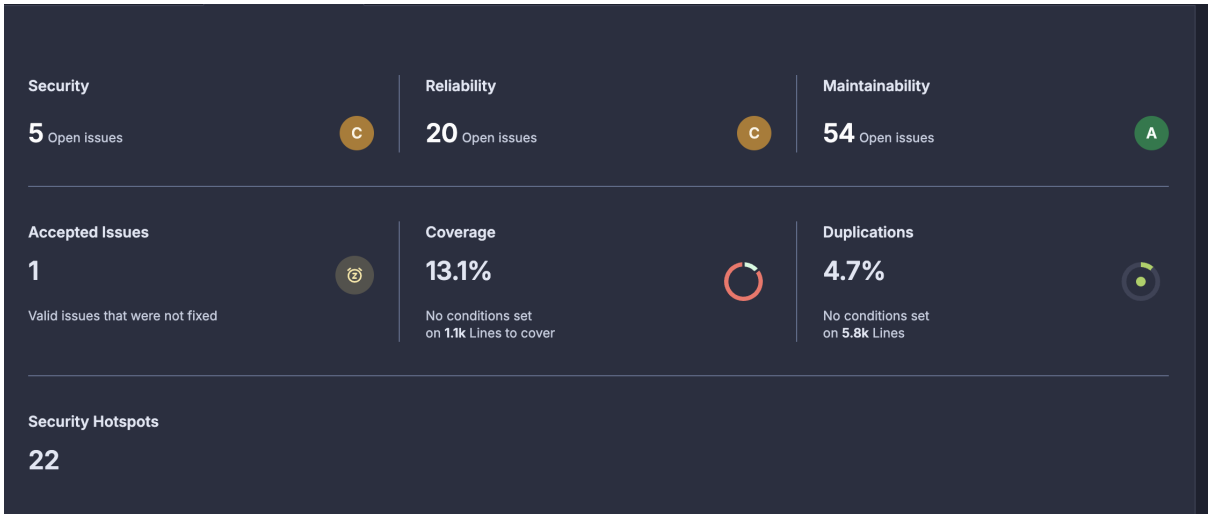

Figure 8: SonarCloud quality gate results.

2.5 Availability and Scaling

TODO : Insert pictures of stucture umls

The application is deployed as a **Docker Swarm** stack with **2 replicas** hosted as 2 seperate **DigtalOcean droplets**. our load balencer distributes traffic across replicas, and automatically reroutes requests away from any node that becomes unavailable/crash. Crashed containers are restarted automatically via the on-failure restart policy.

the application uses a **Redis** instance shared between all nodes which caches data between replica nodes, which ensures consistent returned data regardless of which replica handles the request.
Deployments are made using rolling updates, which starts a new docker container before closing the previous one to avoid downtime. this process is repeated for each node with a 10 second delay to ensure that at least one instance is always available for requests during deployments to reduced downtime of the system.

3 Reflection Perspective

Authors: TODO: names

3.1 Evolution and Refactoring

3.2 Operation

3.3 Maintenance

3.4 DevOps Style of Work

4 Use of Generative AI

Authors: TODO: names

4.1 Tools Used

4.2 Impact on Work Process

References